Discrete Math

"Intro To Graph theory"

Suroj P. Student

Fundamentals

Chapter 1

Key Words: *Define Algorithm and its complexity, Properties of an Algorithm, Binary Search Algorithm, Use Binary Search Algorithm, Insertion Sort Algorithm, Bubble Sort Algorithm*

1.1. Algorithm

There are many general classes of problems that arise in discrete mathematics such as:

- given a sequence of integers, find the largest one
- given a set, list all its subsets
- given a set of integers, put them in increasing order
- given a network, find the shortest path between two vertices. When presented with such a problem
- etc

Definition 1.1 (Algorithm)

An *algorithm* is a finite sequence **precise** instruction for performing a computation or for solving a problem.

♣

Note

Procedure that follows a sequence of steps that leads to the desired answer. Such a sequence of steps is called an **algorithm**

1.1.1. Algorithm for finding the maximum (largest) value in a finite sequence of integers

```
1 | // Find the largest Integer from given get of finite sequence.
2 | Procedure max(a1,a2,a3,...,an: Integer)
3 | max = a1
4 | for i in 2 to n:
5 |   if max < ai then max = ai
6 |
7 | return max[max is the largest integer]
```

1.2. Properties of an Algorithm.

- *Input:* An Algorithm has an input value from a specified set.
- *Output:* From each set of input values an algorithm produces output values from a specified set. The output values are the solution to the problem

- *Definitness:* The step of an Algorithm must be defined precisely.
- *Correctness:* An Algorithm should produce the correct output values for each set of input values.
- *Finiteness:* An algorithm should produce the desired output after a finite (but perhaps large) number of steps for any input in the set.
- *Effectiveness:* It must be possible to perform each step of an algorithm exactly and in a finite amount of time.
- *Generality:* The procedure should be applicable for all problems of the desired form, not just for a particular set of input values.

> **Note**
> To show that the algorithm is correct, we must show that when the algorithm terminates

1.3. *Searching Algorithm.*

- **Problem:** Locating an element in ordered list.
- **Example:** A program that checks the spelling of words searches for them in a dictionary
- **Name of this type of problem:** Searching problems.
- **General form of Searching Problems:** Locate an element x in a list of distinct elements $a_1, a_2, a_3, ..., a_n$ or determine that is not in the list.
- **Solution of Searching Problem:** The location of the term in the list that equal to x (that is, i is the solution if $x = a_i$) and 0 if the x is not in the list.

1.3.1. The Linear Search.

```
1 |   Procedure: linear_search(x:Int,a1,a2,...,an: distinct int)
2 |   i := 1
3 |   while ( i <=n and x = ai)
4 |     i = i+1;
5 |
6 |   if i <= n then location := i
7 |   else location=0
8 |   return location( location of x is i if i or not found if i =0 )
```

1.3.2. Binary Search Algorithm.

- Only applicable for ordered list with **increasing** order.
- **Example:** if the terms are numbers, they are listed from smallest to largest; if they are words, they are listed in lexicographic, or alphabetic, order)
- **Algorithm:**

```
Procedure: binary search (x: int, a1,a2,...,an: increasing integer)
i := 1 ( left endpoint of the search interval)
j := n ( right endpoint of the search interval)
while i < j
    m := floor((i+j)/2)
    if x > am then i := m + 1
    else j := m
if x=ai then location := i
else location := 0
return location[location is the subscript i of the term ai
equal to x or 0 if x not found in list]
```